

DOCUMENTAÇÃO DE SISTEMA

Documentação Técnica

Site institucional, painel de conteúdo e controle de produção da fábrica

Projeto	Borda Tudo — Bordados Computadorizados
Local	Caruaru — PE
Versão	1.21.0 (única para todo o projeto)
Situação	Não publicado. Site travado em construção; o que roda é o /restrito
Base da análise	Código-fonte e banco em C:\Projects\SitesProjects\Borda-Tudo
Data da análise	20 de agosto de 2026

Sumário

1. Visão geral técnica	3
1.1 O que o sistema resolve	3
1.2 As três áreas	3
1.3 Tecnologias	3
2. Estrutura do projeto	5
3. Arquitetura	6
3.1 A trava de construção	6
3.2 Isolamento entre as bases	6
4. Funcionalidades técnicas	8
4.1 Publicação do site	8
4.2 Senha provisória	8
4.3 Tempo real por SSE	8
4.4 O QR das máquinas	8
4.5 Fotos do desenho	9
4.6 O operador nunca vê preço	10
4.7 Recibos e etiquetas	10
4.8 Cifragem	10
5. APIs	11
5.1 API do site e do painel	11
5.2 API do controle de produção	11
5.3 Paginação	12
6. Autenticação e segurança	13
6.1 Papéis	13
6.2 Senhas e sessão	13
6.3 Freio de tentativas	13
6.4 Superfície pública	13
7. Configuração	15
8. Deploy	16
8.1 O portão automático	16
8.2 Operação no servidor	16
8.3 Backup	16
9. Dependências externas	16
10. Tratamento de erros	18

11. Testes 18

12. Pontos de atenção 19

12.1 Documentação existente 19

12.2 Dívidas e limites 19

12.3 O que está notavelmente bem 19

1. Visão geral técnica

1.1 O que o sistema resolve

Bordado se mede em **ponto**. Cada desenho tem uma pontuação fixa — quantos pontos a máquina dá para bordá-lo uma vez — e o que se produz e se cobra é **quantidade × pontuação**.

Hoje o operador anota isso no papel: cliente, desenho, quantidade, pontuação, e soma à mão. Depois o dono copia para a folha dele e marca **um X no canto** para não somar duas vezes. As duas contas são feitas à mão, e o X é o único controle de que a folha já foi lida.

FATO

O sistema existe para tirar as duas contas da mão e o X do canto. Essa frase está escrita no cabeçalho do próprio esquema do banco, e é ela que explica cada decisão de modelagem descrita neste documento: a pontuação vem do cadastro e não da memória de quem anota; o total é calculado pelo banco e nunca digitado; e a ficha é lida uma vez só porque tem identidade, não um X a lápis.

1.2 As três áreas

Área	O que é	Banco	Sessão	Situação
/	Site institucional, páginas estáticas reescritas na publicação	SQLite — data/site.db	—	Travado em construção
/admin/	Painel de conteúdo: textos, fotos, vitrine, situação do site	SQLite — data/site.db	cookie de sessão, 8 h	Funcional
/restrito/	Controle de produção, lotes, notas e caixa	PostgreSQL — bordatudo_producao	cookie rid, 12 h	É o que está no ar

A escolha de dois motores é deliberada e está justificada no código: o /restrito usa PostgreSQL porque é **dado de faturamento** — precisa de transação, de coluna calculada pelo banco e de índice que *impeça* o registro errado de existir. O site continua em SQLite porque é conteúdo, não dinheiro.

1.3 Tecnologias

Camada	Tecnologia	Observação
Runtime	Node.js >= 20	Servidor com node:http, sem framework. Porta 5193, apenas em 127.0.0.1
Banco — site	SQLite via better-sqlite3 ^11.3.0	Com queda controlada para o node:sqlite nativo; WAL ligado
Banco — produção	PostgreSQL via pg ^8.22.0	Papel próprio, sem superusuário
QR das máquinas	qrcode ^1.5.4	Gera a folha de etiquetas para colar no equipamento
Tempo real	Server-Sent Events	Nativo, sem biblioteca
Frontend	HTML, CSS e JS sem framework	restrito/app.html (306 KB) e admin/index.html (39 KB), cada um em arquivo único
Proxy / serviço	nginx + systemd	bordatudo.service, configuração versionada
Entrega	GitHub Actions	Portão de prova + entrega por ssh restrito

Três dependências de produção, e só. Não há dependências de desenvolvimento: as suítes de teste usam apenas o que o Node traz.

FATO

O `db.js` abre um banco **de mentira em memória** logo após o `require` do `better-sqlite3`. Não é excesso de zelo: o `require` carrega só JavaScript e passa mesmo quando o binário nativo não serve para aquele Node. O erro real (`ERR_DLOPEN_FAILED`) só apareceria uma linha depois, já fora do `try` — e o desvio que existe para manter o site de pé nunca dispararia justamente no caso mais provável. Aconteceu na primeira subida deste projeto.

2. Estrutura do projeto

Caminho	Tamanho	Conteúdo
restrito.js	211 KB	O sistema de produção inteiro: rotas, papéis, jornada, fichas, lotes, amálgama, notas, caixa, recibos e etiquetas
server.js	60 KB	Site público, publicação das páginas, painel /admin, situação do site, contador de visitas
pg.js	16 KB	Camada do PostgreSQL: tradução de parâmetro, transações, decifragem na leitura
db.js	2,7 KB	Único lugar que abre SQLite; escolhe o driver
cripto.js	7,5 KB	Cifragem de dados sensíveis
html-seguro.js	12 KB	Sanitizador de HTML por lista de permitidos
limitador.js	8,5 KB	Freio de tentativas de senha
backup.js	11 KB	Cópia dos dois bancos, dentro do processo
criar-usuario.cjs	9 KB	CLI de contas, inclusive a de dono
sql/	10 arquivos	Esquema do PostgreSQL, numerado e idempotente
src/	8 moldes + 6 scripts	Modelos das páginas e geradores de logo/favicon
restrito/app.html	306 KB	Interface completa da produção
admin/index.html	39 KB	Painel de conteúdo
servicos/, vitrine/, empresa/, orcamento/, privacidade/, busca/	6 pastas	Páginas geradas pela publicação — não editar à mão
index.html	33 KB	Home, também gerada
construcao.html manutencao.html 404.html	3 páginas	Avisos, com CSS embutido
testar-restrito.cjs	121 KB	Suíte da produção
testar-financeiro.cjs	31 KB	Suíte do dinheiro
testar-site.cjs	13 KB	Suíte da situação do site
deploy.sh, verificar.sh, restaurar.sh	3 scripts	Operação no servidor
README.md, SUBIR.md	2 documentos	Visão geral e roteiro de instalação
data/ (*)	site.db, desenhos/	Banco do site e fotos dos desenhos
backups/ (*)	17 arquivos	Cópias automáticas dos dois bancos

(*) não versionado.

PONTO DE ATENÇÃO - o que é gerado

As pastas de página são saída, não fonte. O que se edita são os moldes em `src/`; a publicação reescreve `index.html` e as seis pastas a partir deles. Alterar uma página gerada funciona até a próxima publicação.

3. Arquitetura

Um processo Node, duas bases, três superfícies. O que garante que a queda de uma não leve a outra é um catch em volta da delegação para o /restrito.

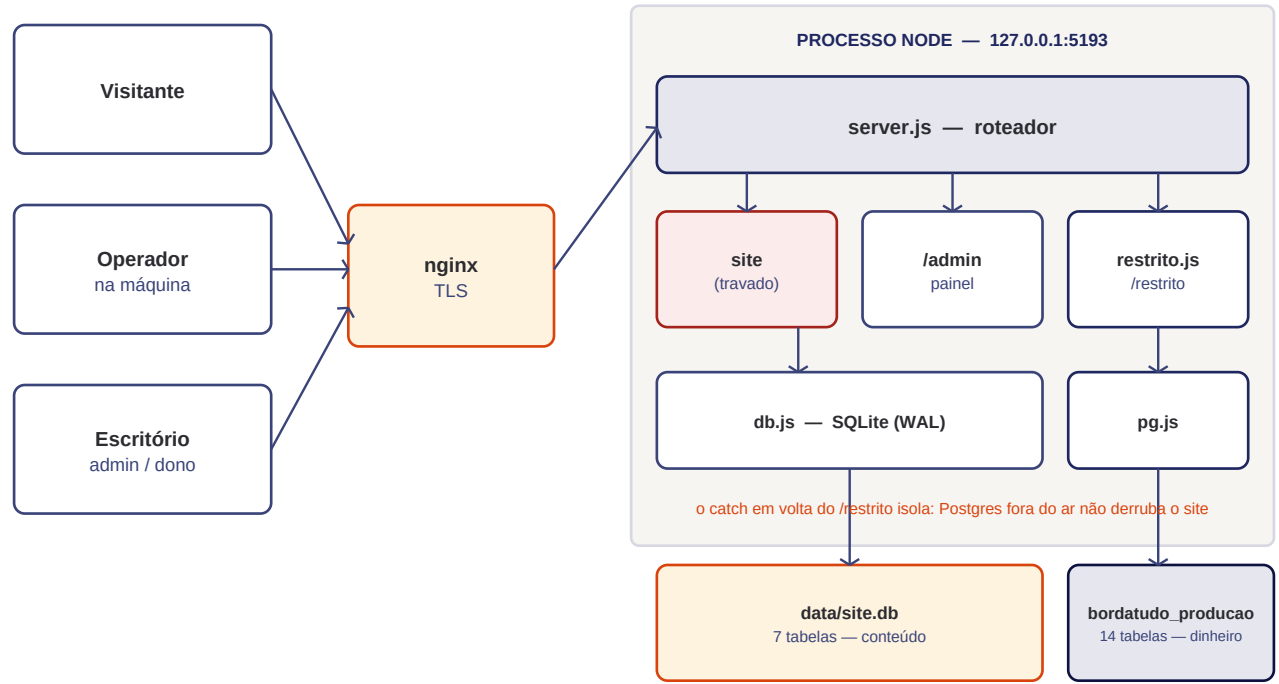


Figura 1 — Arquitetura de execução.

3.1 A trava de construção

O site tem três estados no painel — **no ar**, **em construção** e **em manutenção** — e **acima dos três** existe uma constante no código: TRAVA_CONSTRUCAO, hoje em true.

Camada	Onde	Efeito
Trava	Constante em server.js	Enquanto for true, escolher “Site no ar” no painel não abre o site. Ganha de tudo
Estado	settings.site_estado	no-ar · construcao · manutencao
Resposta	HTTP 503	Com Retry-After de 86400 s em construção e 600 s em manutenção
Exceções	server.js	/admin, /assets e o favicon continuam servindo — o painel precisa abrir, e uma aba com ícone quebrado parece defeito, não obra

FATO

Por que constante e não configuração: **configuração se muda sem querer**. E a distinção entre os dois estados não é enfeite — manutenção *promete que o site volta*, e prometer volta de um site que nunca subiu é a primeira impressão que a empresa dá. O painel recebe travaConstrucao: true na API justamente para poder avisar; sem isso mostraria “Site no ar” selecionado com o site fora do ar.

RECOMENDAÇÃO

Para lançar: false na constante, “Site no ar” no painel e deploy — nessa ordem. São três passos porque a trava é intencionalmente redundante.

3.2 Isolamento entre as bases

O `server.js` chama `restrito.rotas(...)` dentro de um `catch`. Uma falha do PostgreSQL vira erro daquela requisição, não do processo: o site e o painel, que vivem no SQLite, continuam respondendo. É o mesmo raciocínio do `require` protegido dos drivers.

4. Funcionalidades técnicas

4.1 Publicação do site

São **8 moldes** em `src/` que geram **7 páginas**. O conteúdo vem de **58 chaves** de texto em settings mais quatro listas em tabelas próprias (serviços, peças da vitrine, depoimentos e dúvidas).

Mecanismo	Papel
<code>trocarTokens / setMarker</code>	Substituem valores e blocos delimitados dentro do molde
<code>montarServicos, montarVitrine, montarDepoimentos, montarDuvidas</code>	Geram as listas a partir do banco
<code>montarJsonLd</code>	Dados estruturados da empresa
<code>montarSelos</code>	Os quatro números de destaque da home
<code>carimbar</code>	Marca as páginas com a versão gerada
<code>escreverSitemap</code>	Reescreve o <code>sitemap.xml</code> a cada publicação
<code>linhaUnica</code>	Achata blocos que não podem quebrar linha

4.2 Senha provisória

Toda conta nasce — e toda redefinição gera — uma senha de **6 dígitos**, fácil de ditar por telefone. Ela serve para **abrir a porta uma vez**.

- A coluna `usuarios.senha_provisoria` é o que transforma a intenção em regra: enquanto for verdadeira, a sessão fica **trancada no servidor** — só passam `/eu, /eu/senha` e `/sair`.
- Na primeira troca **não se pede a senha atual** (quem cadastrou a conhece); na troca voluntária, sim.
- Quem já estava no sistema quando a coluna nasceu **não** foi obrigado a trocar: essas senhas já haviam sido escolhidas por quem as usa.

FATO

A conta de **dono** é a exceção: a senha dela nasce definitiva de propósito. Com `senha_provisoria` ela ficaria trancada para fora, porque a própria rota de trocar senha recusa a conta de dono — ela só é alterada pelo terminal do servidor.

4.3 Tempo real por SSE

`GET /restrito/api/eventos` mantém uma conexão aberta por sessão. A escolha por **Server-Sent Events** em vez de WebSocket está justificada: o fluxo é de mão única e o `EventSource` reconecta sozinho.

FATO

O evento leva só o assunto, nunca o dado. É o que impede o preço de chegar ao navegador do operador por um caminho lateral — o mesmo cuidado que o filtro de segredos aplica nas respostas comuns. A tela recebe “mudou lote” e vai buscar o que tem direito de ver.

PONTO DE ATENÇÃO - limite conhecido

A lista de conexões vive na **memória do processo**. Com dois processos servindo o mesmo sistema, o evento nasceria num e não chegaria ao outro. A correção prevista é `LISTEN/NOTIFY` do PostgreSQL. Hoje o serviço roda em processo único, então não é um defeito ativo — é um limite a respeitar antes de escalar.

4.4 O QR das máquinas

Cada máquina tem um token aleatório que vai dentro de um QR code colado no equipamento. Escanear abre a tela do operador já sabendo em qual máquina ele está. A rota `/restrito/etiquetas` gera a folha para

impressão.

FATO

O token identifica, não autentica. Está impresso num adesivo à vista de todos — qualquer pessoa que passe pela fábrica fotografa. Tratá-lo como senha seria dar acesso ao sistema a quem tem uma câmera. Quem entra continua sendo o operador, com o login dele; o QR só responde “qual máquina”. Por isso trocar o token serve apenas para **invalidar adesivos antigos**, não para expulsar ninguém — e desativar uma máquina rotaciona o token automaticamente.

4.5 Fotos do desenho

Decisão	Motivo
Gravadas em data/desenhos/, nunca em assets/	Em assets bastaria acertar o nome do arquivo para baixar o bordado de qualquer cliente
Servidas por /restrito/foto/<arquivo>, com sessão	O nome do arquivo é validado por expressão restrita — sem barras, sem caminho relativo
Aceitas por assinatura do arquivo , não por extensão	Renomear um script para .png não o transforma em imagem
Tabela própria (desenho_fotos), não coluna	Uma coluna obrigaria a escolher uma foto; uma lista dentro de uma coluna tornaria impossível apagar UMA sem reescrever as outras — e é aí que se perde a errada

4.6 O operador nunca vê preço

O preço do desenho é segredo do escritório. A proteção acontece em **dois sentidos**, num lugar só:

Função	Direção	O que faz
esconderSegredos	Saída	Remove as colunas marcadas como soAdmin de toda linha devolvida a quem não é admin
tirarSegredosDoCorpo	Entrada	Remove as mesmas colunas do corpo da requisição — senão o operador poderia gravar um preço que o escritório não escolheu

FATO

A saída importa porque são **cinco consultas** que devolvem desenho e todas nasceram com `SELECT d.*`. A tela não mostra o campo — mas o corpo da resposta o entregaria a quem abrisse as ferramentas do navegador. Filtrar na consulta, uma a uma, deixaria a sexta consulta sem proteção no dia em que ela fosse escrita.

O operador **cadastra** desenho (cria e anexa imagem; não altera nem exclui). Sem isso ele penduraria a produção num desenho parecido, e o relatório ficaria errado sem rastro. O desenho nasce sem preço e cai no filtro `?sem_preco=1` do admin — que tem índice parcial próprio no banco.

4.7 Recibos e etiquetas

O `restrito.js` gera dois documentos para impressão, em HTML montado no servidor: **recibo do lote** (`/restrito/lotos/:id/recibo`) e **recibo de pagamento** (`/restrito/lancamentos/:id/recibo`), além da folha de **etiquetas QR**. São cerca de 700 linhas dedicadas a papel — o que confirma que a saída impressa é parte da entrega, não um extra.

4.8 Cifragem

`cripto.js` existe e é consumido por `pg.js`, que decifra na leitura — mesmo desenho usado em outros projetos do parque. A chave vem de `DADOS_CHAVE` no ambiente.

INFERÊNCIA

No estado atual do esquema **nenhuma coluna é declarada como cifrada** neste projeto — a infraestrutura está montada e ligada, mas o conjunto de campos protegidos está vazio. Faz sentido para o domínio: a base guarda cliente, desenho, quantidade e valor, não documento nem dado de saúde. A camada está pronta caso um campo passe a exigí-la.

5. APIs

Duas APIs em JSON, sem framework de roteamento: as rotas são comparações e expressões regulares sobre o caminho.

5.1 API do site e do painel

Método	Rota	Finalidade
POST	/api/entrar	Autentica no painel (senha única)
POST	/api/sair	Encerra a sessão
GET	/api/conteudo	Todo o conteúdo editável + travaConstrucao
PUT	/api/conteudo	Grava textos (só chaves conhecidas passam)
—	listas de serviços, vitrine, depoimentos e dúvidas	CRUD das quatro tabelas de conteúdo
POST	/api/publicar	Regenera as 7 páginas e o sitemap
GET	/api/acessos	Contagem de visitas
POST	/api/senha	Troca a senha do painel
—	backup	Dispara e consulta a cópia dos bancos

Valor fora da lista de chaves conhecidas é **ignorado** na gravação: um estado inválido em `site_estado` tiraria o site do ar sem página nenhuma para mostrar, e o painel continuaria dizendo que está tudo certo.

5.2 API do controle de produção

Sessão e conta

Método	Rota	Finalidade
POST	/restrito/api/entrar	Autentica. Trava por IP após 5 erros em 15 min
POST	/restrito/api/sair	Encerra a sessão
GET	/restrito/api/eu	Quem sou, papel e se sou admin
PUT	/restrito/api/eu/senha	Troca a própria senha
GET	/restrito/api/eventos	SSE — fluxo de avisos
GET	/restrito/api/maquina-do-qr	Resolve o token do QR em máquina

Produção

Método	Rota	Finalidade
POST	/restrito/api/jornadas	Início de produção
POST	/restrito/api/jornadas/:id/encerrar	Fim do expediente
GET	/restrito/api/meu-dia	O dia do operador: horas e fichas
GET	/restrito/api/meu-mes	Calendário de horas do mês
POST	/restrito/api/fichas	Abrir ficha (cliente, desenho, máquina)
PUT	/restrito/api/fichas/:id	Corrigir a ficha
POST	/restrito/api/fichas/:id/fechar	Fechar ficha (quantidade, mercadoria, cor)
POST	/restrito/api/fichas/:id/cancelar	Cancelar
DELETE	/restrito/api/fichas/:id	Remover — admin, com três cercas
POST	/restrito/api/fichas/somar	Somar fichas de um lote numa só

Método	Rota	Finalidade
GET	/restrito/api/producao	Painel da produção, com filtros e paginação
GET	/restrito/api/horarios	Horas de todos os operadores
PUT	/restrito/api/horarios/:id	Corrigir jornada

Lotes, notas e caixa

Método	Rota	Finalidade
GET / POST	/restrito/api/lotes	Lista e cria lotes
GET / PUT	/restrito/api/lotes/:id	Detalhe e edição (situação, nota, pago)
PUT	/restrito/api/lotes/:id/fichas	A amálgama — recebe a lista completa e reconcilia
GET	/restrito/api/clientes/:id/lotes fichas	As “gavetas” do cliente
GET / POST	/restrito/api/notas	Lista e emite nota
GET / PUT / DELETE	/restrito/api/notas/:id	Detalhe, edição e exclusão
PUT	/restrito/api/notas/:id/lotes	Quais lotes entram na nota
POST	/restrito/api/lancamentos	Lança dinheiro (entrada ou saída)
POST	/restrito/api/lancamentos/:id/cancelar	Cancela — a linha permanece
GET	/restrito/api/caixa	Extrato do caixa
GET	/restrito/lotes/:id/recibo	Recibo do lote, em HTML para impressão
GET	/restrito/lancamentos/:id/recibo	Recibo de pagamento

Cadastros e administração

Método	Rota	Finalidade
GET/POST/PUT/DELETE	/restrito/api/{clientes desenhos mercadorias cores maquinas}/{/:id}	CRUD dos cinco cadastros
PUT	/restrito/api/{cadastro}/{:id}/ativo	Desativar — rota separada da exclusão
GET/POST/DELETE	/restrito/api/desenhos/:id/fotos[:foto]	Fotos do desenho
GET	/restrito/foto/:arquivo	Serve a imagem, com sessão
POST	/restrito/api/maquinas/:id/qr/trocar	Rotaciona o token do QR
GET	/restrito/etiquetas	Folha de etiquetas para impressão
GET/POST	/restrito/api/usuarios	Contas (admin)
PUT/DELETE	/restrito/api/usuarios/:id	Editar e excluir conta
PUT	/restrito/api/usuarios/:id/senha	Redefinir senha — gera provisória

FATO

Excluir e desativar são rotas diferentes de propósito (DELETE × PUT .../ativo). Um botão só, que “apaga se puder e desativa se não”, esconderia do usuário qual das duas coisas aconteceu — e as duas têm consequências opostas no histórico.

5.3 Paginação

As listas aceitam ?pagina= e devolvem um envelope com o total. **Sem o parâmetro, a rota devolve tudo** — e isso é intencional: é o que os seletores da tela do operador consomem. O teto por página é de 500 registros, ajustável por variável de ambiente.

6. Autenticação e segurança

6.1 Papéis

Papel	Alcance	Particularidades
dono	Tudo, acima de admin	Invisível na lista e responde 404 em todas as rotas de usuário. Só uma conta (índice único). Criada apenas pelo terminal, com senha longa e definitiva
admin	Produção, lotes, notas, caixa, cadastros e contas	Vê preço e valor
operador	Abrir e fechar as próprias fichas; cadastrar desenho	Nunca vê preço . Não altera nem exclui desenho

FATO

Esconder não é proteger. A conta de dono responde 404 — e não 403 — nas rotas de usuário, porque o id 1 é o primeiro palpite de quem procura. Um 403 confirmaria que a conta existe.

FATO

`ehAdmin()` é a **única porta**. Havia oito comparações espalhadas do tipo `papel !== "admin"`; cada uma que sobrevivesse trancaria o dono para fora justamente da tela que ele existe para consertar. A tela decide por `EU.admin` vindo do servidor, não pelo nome do papel.

6.2 Senhas e sessão

Aspecto	Implementação
Derivação	scrypt N=16384, r=8, p=1, 32 bytes, com sal por senha
Comparação	Tempo constante
Isca	Hash descartável verificado quando o usuário não existe, para o tempo de resposta não denunciar contas válidas
Sessão	Cookie <code>rid</code> de 48 hexadecimais, 12 h no /restrito e 8 h no painel do site
Marca Secure	Aplicada quando a requisição chegou por HTTPS
Senha gerada	<code>senhaDitavel()</code> , compartilhada entre a tela e o CLI — o mesmo formato nos dois caminhos

6.3 Freio de tentativas

Cinco erros em 15 minutos travam o **IP**. O estado é persistido em arquivo, então reiniciar o serviço não zera o freio.

PONTO DE ATENÇÃO - risco operacional conhecido

Na fábrica todo mundo sai pelo mesmo IP público. Se algumas pessoas errarem a senha na mesma manhã, a loja inteira fica de fora — e a resposta é **429**, não 401, o que torna o sintoma pouco óbvio para quem está na máquina. O risco está reconhecido no README. Se doer, o caminho é afrouxar o limite por IP mantendo o balde por conta.

6.4 Superfície pública

- A aplicação escuta **apenas em 127.0.0.1**; quem fala com o mundo é o nginx.
- Lista de **permitidos** para servir arquivo: só as pastas do site e os arquivos declarados. O restante não é servido — nem por engano de caminho.
- Fotos de desenho ficam fora de `assets/` e exigem sessão.
- Todo HTML vindo de campo de texto passa por `sanitizarHtml`, por lista de tags permitidas.

PONTO DE ATENÇÃO - pendência de segurança confirmada

O sanitizador não está ligado no editor do painel do site. `sanitizarHtml` é chamado em **12 pontos** do `restrito.js` e em **nenhum** do `server.js` — confirmado por varredura. Como o painel tem editor com negrito, itálico e link, o conteúdo digitado ali entra nas páginas geradas sem passar pelo filtro. O risco é limitado (quem edita já é o administrador, e o site está fora do ar), mas a defesa existe no projeto e não está ligada onde deveria.

7. Configuração

Variável	Obrigatória	Padrão	Para que serve
PGPASSWORD	sim	—	Senha do papel do PostgreSQL
DADOS_CHAVE	sim	—	32 bytes em base64 para a camada de cifragem
PGHOST · PGPORT · PGUSER · PGDATABASE	não	127.0.0.1 · 5432 · bordatudo · bordatudo_producao	Conexão
PORT	não	5193	Porta do processo
HOST	não	127.0.0.1	Interface de escuta
SITE_DB	não	data/site.db	Banco do site — é o que permite a suíte rodar contra um descartável
LIMITES_ARQUIVO	não	data/limites.json	Arquivo do freio de tentativas
TETO_POR_PAGINA	não	500	Teto da paginação
PORTA_TESTE · PORTA_TESTE_SITE · PORTA_TESTE_FIN	não	5199 · 5198 · 5197	Portas das três suítes

Em produção, as credenciais vivem em `/etc/bordatudo.env`, carregado pelo `systemd` com `EnvironmentFile=-.`. Localmente, um `.env` na raiz — que não é versionado.

FATO

O arquivo `sql/01-criar-banco.sql` traz uma senha de exemplo **visível de propósito**, com a instrução de trocá-la antes de rodar. O comentário explica: senha em arquivo versionado é senha pública, e o arquivo é o roteiro, não o cofre.

8. Deploy

8.1 O portão automático

O pipeline tem dois trabalhos em sequência: **provar** e **entregar**. O segundo só existe se o primeiro passar.

Duas particularidades deste projeto que o portão precisou respeitar, e estão escritas no próprio arquivo:

1. **O site responde 503 de propósito.** Um portão que exigisse 200 na raiz reprovava o estado desejado.
2. **O esquema vem de arquivos .sql numerados**, não de uma ferramenta de migração. O portão levanta um PostgreSQL descartável e aplica os arquivos na ordem.

FATO

Uma correção registrada no histórico (1.14.1) foi exatamente sobre isso: *a CI montava o esquema por uma lista escrita à mão*. Uma lista paralela à pasta significa que um arquivo novo pode não entrar — e o portão passaria a provar um esquema diferente do que vai para produção.

8.2 Operação no servidor

```
./verificar.sh      # só lê: versão, bancos, serviço, permissões
sudo ./deploy.sh   # pg_dump -> pull -> npm ci -> migrações -> restart
sudo ./restaurar.sh # restauração assistida
```

Regra	Motivo
O deploy.sh faz pg_dump antes do pull e para se o dump falhar	Atualizar sem cópia do banco de faturamento é o passo em que não se pode voltar
As migrações rodam depois do pull	O código novo espera o esquema novo
bordatudo.service usa After=postgresql, nunca Requires	A unit do PostgreSQL é fachada: Requires amarraria o serviço a um estado que não reflete o banco de verdade
EnvironmentFile=-/etc/bordatudo.env	O hífen faz o serviço subir mesmo sem o arquivo, em vez de falhar de forma obscura

O roteiro completo de instalação está em SUBIR.md, com **10 passos** e o que conferir em cada um — do PostgreSQL à impressão dos QR das máquinas.

8.3 Backup

Banco	Método	Observação
data/site.db	VACUUM INTO	Backup online do SQLite. O caminho é normalizado com barras para funcionar no Windows, e há desempate por sufixo porque o comando se recusa a sobrescrever
bordatudo_producao	pg_dump	Arquivo de texto, não cópia do diretório do PostgreSQL. Dump vazio é tratado como erro — o comando pode devolver zero e um arquivo vazio, que só se descobre na hora de restaurar

PONTO DE ATENÇÃO - pendência

O **backup existe só no mesmo disco**. Cobre erro humano e engano de operação, não perda da máquina. É a pendência operacional mais relevante do projeto, e está reconhecida no README.

9. Dependências externas

Serviço	Uso	Se faltar
PostgreSQL	Todo o /restrito	O /restrito para; o site e o painel continuam
pg_dump	Backup da produção	O backup do site continua; o da produção falha com aviso

Serviço	Uso	Se faltar
WhatsApp (wa.me)	Canal de contato do site e da página de construção	Link quebrado
Google Maps	Endereço no site	Bloco vazio
Medição (analytics, pixel)	Campos no painel	Não configurados — os dois estão vazios

Não há chamada de API externa em tempo de execução: nenhuma consulta de CEP, nenhum serviço de terceiro no caminho de uma requisição. O sistema funciona com a máquina e o banco.

10. Tratamento de erros

Situação	Comportamento
Falha no /restrito	Capturada por catch no roteador; vira erro daquela requisição. O site não cai junto
Erro do banco	erroDeBanco() traduz o código do PostgreSQL para mensagem de negócio — violação de índice único vira “já existe”, não “erro 23505”
Vínculo impedindo exclusão	409 com a lista do que segura (“está em 14 fichas”), e a sugestão de desativar
Driver nativo incompatível	Queda para o driver de fábrica, com aviso no boot
Dump de backup vazio	Tratado como erro , não sucesso
Porta ocupada na suíte	A suíte se recusa a rodar e diz o que fazer — senão rodaria contra outro processo e o erro apareceria longe da causa
Sessão com senha provisória	Bloqueada no servidor em todas as rotas menos três
Site fora do ar	503 com Retry-After, e não 200 numa página de aviso — evita que o buscador indexe o aviso no lugar da home

11. Testes

Três suítes, todas versionadas, executadas por `npm test`. Não há framework: são scripts Node que sobem o servidor de verdade e conversam por HTTP.

Suíte	Porta	Conferências	Alcance
testar-restrito.cjs	5199	501	Papéis, jornada, fichas, amálgama, somar, lotes, cadastros, vínculos, paginação, tempo real, conta de dono, força bruta
testar-financeiro.cjs	5197	84	Notas, lançamentos, caixa, salário com dono, estados impossíveis recusados pelo banco, operador sem acesso ao dinheiro
testar-site.cjs	5198	47	Trava de construção, os três estados, troca pelo painel, migração de chave antiga

FATO

Executadas nesta análise, em 20/08/2026: 632/632 verdes (501 + 84 + 47). As contagens do banco antes e depois são idênticas — as suítes criam com prefixo `ZZ_QA` e apagam por id, nunca por nome.

Prova por sabotagem. O projeto registra 12 quebras propositalis que derrubam a suíte. Duas lições ficaram documentadas: o alvo da sabotagem precisa ser **único no arquivo** (uma expressão que existia em duas rotas fez a troca pegar a errada — e de brinde revelou uma guarda sem teste nenhum), e **trocar só a mensagem não desliga a guarda**.

PONTO DE ATENÇÃO - colisão de portas

As portas padrão das suítes colidem com produção de outros projetos do parque: 5198 é a porta do Izatec e 5197 é a do LA Chat. A suíte se protege — recusa rodar se a porta estiver ocupada —, mas isso a trava sempre que o vizinho está no ar. Contorno imediato: as variáveis `PORTA_TESTE_SITE` e `PORTA_TESTE_FIN`.

RECOMENDAÇÃO

Mudar os padrões para uma faixa reservada a teste (por exemplo 53xx), em vez de depender de variável de ambiente lembrada na hora. É alteração de três constantes.

12. Pontos de atenção

12.1 Documentação existente

O README.md tem 358 linhas e é **de qualidade acima da média**: explica o domínio, a instalação, o QR das máquinas e as armadilhas. Mas cobre o sistema até por volta da versão 1.12 e **não acompanhou os módulos seguintes**.

Assunto	Existe no código?	Está no README?
Fichas, lotes, amálgama, cadastros, QR, dono, SSE	sim	sim, bem descrito
Financeiro: notas, lançamentos, caixa, recibos	sim, com suite própria	não — duas menções soltas a "financeiro", nenhuma a lançamento
Somar fichas de um lote	sim, com a situação somada	não
Expediente do operador e tela de Horários	sim, com regra em JSONB	não
Data retroativa do bordado e valor da ficha	sim	não
Versão e histórico	1.21.0 em package.json	não menciona versão nenhuma

RECOMENDAÇÃO

O README foi complementado nesta entrega com uma seção sobre o financeiro, as horas e a soma de fichas, além do índice para os documentos em docs/. A recomendação de processo é a mesma dos demais projetos: **atualizar o README faz parte de subir a versão**.

NÃO IDENTIFICADO

Não existe CHANGELOG. O histórico de versões só vive nas mensagens de commit, e as cinco últimas entregas foram registradas como chore: ajustes — sem número de versão, ao contrário das anteriores, que traziam "1.15.0 — calendário de horas...". Isso torna difícil reconstruir o que entrou em cada versão a partir do repositório.

12.2 Dívidas e limites

#	Item	Natureza	Gravidade
1	Backup só no mesmo disco — perda da máquina é perda de tudo	Risco operacional	Alta
2	Sanitizador não ligado no painel do site — 12 chamadas no /restrito, zero no server.js	Segurança	Média
3	Portas de teste colidem com produção de outros projetos	Qualidade	Média
4	Freio por IP trava a fábrica inteira , que sai por um IP só	Risco operacional	Média
5	SSE em memória — impede rodar em mais de um processo	Limite de escala	Baixa hoje
6	CNPJ ainda é "0" no painel — não sai no rodapé nem no recibo	Dado pendente	Média
7	Sem CHANGELOG e commits recentes sem número de versão	Processo	Baixa
8	app.html com 306 KB em arquivo único, com estrutura, estilo e comportamento juntos	Dívida técnica	Baixa
9	SEO e integrações não configurados (medição vazia)	Pendente até o lançamento	Baixa
10	Nunca publicado — nenhuma parte foi exercitada em servidor real	Risco de estreia	Média

12.3 O que está notavelmente bem

Um documento de análise que só lista problemas dá uma imagem falsa. Três pontos merecem registro:

- **As regras de negócio estão no banco, não só na tela**. Coluna gerada, índice único parcial e 23 restrições de verificação tornam o estado errado *impossível*, em vez de improvável. É o oposto do padrão mais comum, em

que o banco é um depósito e toda regra vive na aplicação.

- **Os arquivos SQL explicam o porquê, não só o quê.** Cada decisão de modelagem vem com o erro que ela evita — inclusive as decisões de *não* fazer algo, como não guardar total no lote.
- **A cobertura de teste é real e provada por sabotagem.** 632 conferências que sobem o servidor e conversam por HTTP, com quebras propositais registradas para demonstrar que cada guarda é de fato exercitada.

FATO

Nota de método. Todo endpoint, tabela, coluna, restrição e índice citado aqui foi lido no código ou consultado **no banco em execução**. As três suítes foram executadas durante esta análise. Nenhuma senha, chave ou credencial real foi reproduzida. O projeto estava com a árvore de trabalho limpa no commit f48f1e3.